

The quicksort algorithm has a worst-case running time of $\Theta(n^2)$ on an input array of n numbers. Despite this slow worst-case running time, quicksort is often the best practical choice for sorting because it is remarkably efficient on average: its expected running time is $\Theta(n \lg n)$ when all numbers are distinct, and the constant factors hidden in the $\Theta(n \lg n)$ notation are small. Unlike merge sort, it also has the advantage of sorting in place (see page 158), and it works well even in virtual-memory environments.

Our study of quicksort is broken into four sections. Section 7.1 describes the algorithm and an important subroutine used by quicksort for partitioning. Because the behavior of quicksort is complex, we'll start with an intuitive discussion of its performance in Section 7.2 and analyze it precisely at the end of the chapter. Section 7.3 presents a randomized version of quicksort. When all elements are distinct,¹ this randomized algorithm has a good expected running time and no particular input elicits its worst-case behavior. (See Problem 7-2 for the case in which elements may be equal.) Section 7.4 analyzes the randomized algorithm, showing that it runs in $\Theta(n^2)$ time in the worst case and, assuming distinct elements, in expected $O(n \lg n)$ time.

7.1 Description of quicksort

Quicksort, like merge sort, applies the divide-and-conquer method introduced in Section 2.3.1. Here is the three-step divide-and-conquer process for sorting a subarray $A[p : r]$:

Divide by partitioning (rearranging) the array $A[p : r]$ into two (possibly empty) subarrays $A[p : q - 1]$ (the *low side*) and $A[q + 1 : r]$ (the *high side*) such that each element in the low side of the partition is less than or equal to the *pivot* $A[q]$, which is, in turn, less than or equal to each element in the high side. Compute the index q of the pivot as part of this partitioning procedure.

Conquer by calling quicksort recursively to sort each of the subarrays $A[p : q - 1]$ and $A[q + 1 : r]$.

Combine by doing nothing: because the two subarrays are already sorted, no work is needed to combine them. All elements in $A[p : q - 1]$ are sorted and less than or equal to $A[q]$, and all elements in $A[q + 1 : r]$ are sorted and greater than or equal to the pivot $A[q]$. The entire subarray $A[p : r]$ cannot help but be sorted!

The QUICKSORT procedure implements quicksort. To sort an entire n -element array $A[1 : n]$, the initial call is QUICKSORT ($A, 1, n$).

```
QUICKSORT( $A, p, r$ )
1  if  $p < r$ 
2      // Partition the subarray around the pivot, which ends up in  $A[q]$ .
3       $q = \text{PARTITION}(A, p, r)$ 
4      QUICKSORT( $A, p, q - 1$ )    // recursively sort the low side
5      QUICKSORT( $A, q + 1, r$ )    // recursively sort the high side
```

Partitioning the array

The key to the algorithm is the PARTITION procedure on the next page, which rearranges the subarray $A[p : r]$ in place, returning the index of the dividing point between the two sides of the partition.

Figure 7.1 shows how PARTITION works on an 8-element array. PARTITION always selects the element $x = A[r]$ as the pivot. As the procedure runs, each element falls into exactly one of four regions, some of which may be empty. At the start of each iteration of the **for** loop in lines 3–6, the regions satisfy certain properties, shown in Figure 7.2. We state these properties as a loop invariant:

```

PARTITION( $A, p, r$ )
1  $x = A[r]$                                 // the pivot
2  $i = p - 1$                                 // highest index into the low side
3 for  $j = p$  to  $r - 1$                       // process each element other than the
                                           // pivot
4     if  $A[j] \leq x$                           // does this element belong on the low
                                           // side?
5          $i = i + 1$                             // index of a new slot in the low side
6         exchange  $A[i]$  with  $A[j]$            // put this element there
7 exchange  $A[i + 1]$  with  $A[r]$              // pivot goes just to the right of the low
                                           // side
8 return  $i + 1$                                 // new index of the pivot

```

At the beginning of each iteration of the loop of lines 3–6, for any array index k , the following conditions hold:

1. if $p \leq k \leq i$, then $A[k] \leq x$ (the tan region of Figure 7.2);
2. if $i + 1 \leq k \leq j - 1$, then $A[k] > x$ (the blue region);
3. if $k = r$, then $A[k] = x$ (the yellow region).

We need to show that this loop invariant is true prior to the first iteration, that each iteration of the loop maintains the invariant, that the loop terminates, and that correctness follows from the invariant when the loop terminates.

Initialization: Prior to the first iteration of the loop, we have $i = p - 1$ and $j = p$. Because no values lie between p and i and no values lie

between $i + 1$ and $j - 1$, the first two conditions of the loop invariant are trivially satisfied. The assignment in line 1 satisfies the third condition.

Maintenance: As Figure 7.3 shows, we consider two cases, depending on the outcome of the test in line 4. Figure 7.3(a) shows what happens when $A[j] > x$: the only action in the loop is to increment j . After j has been incremented, the second condition holds for $A[j - 1]$ and all other entries remain unchanged. Figure 7.3(b) shows what happens when $A[j] \leq x$: the loop increments i , swaps $A[i]$ and $A[j]$, and then increments j . Because of the swap, we now have that $A[i] \leq x$, and condition 1 is satisfied. Similarly, we also have that $A[j - 1] > x$, since the item that was swapped into $A[j - 1]$ is, by the loop invariant, greater than x .

Termination: Since the loop makes exactly $r - p$ iterations, it terminates, whereupon $j = r$. At that point, the unexamined subarray $A[j : r - 1]$ is empty, and every entry in the array belongs to one of the other three sets described by the invariant. Thus, the values in the array have been partitioned into three sets: those less than or equal to x (the low side), those greater than x (the high side), and a singleton set containing x (the pivot).

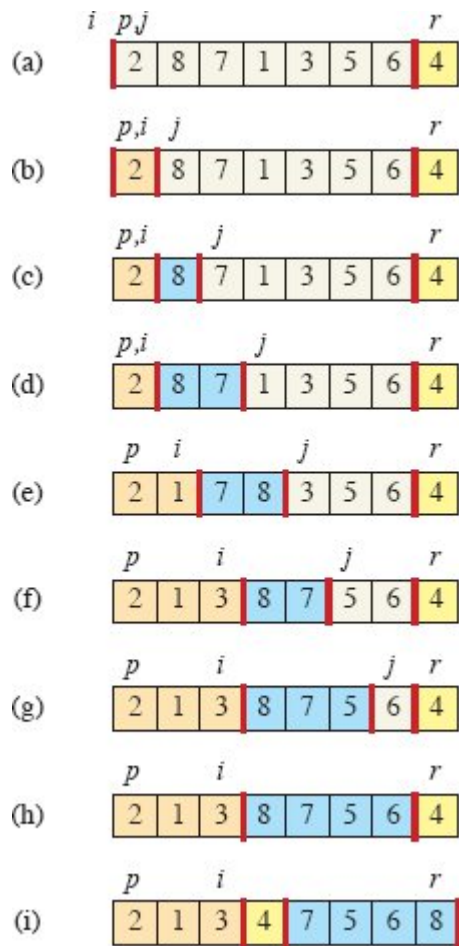


Figure 7.1 The operation of PARTITION on a sample array. Array entry $A[r]$ becomes the pivot element x . Tan array elements all belong to the low side of the partition, with values at most x . Blue elements belong to the high side, with values greater than x . White elements have not yet been put into either side of the partition, and the yellow element is the pivot x . (a) The initial array and variable settings. None of the elements have been placed into either side of the partition. (b) The value 2 is “swapped with itself” and put into the low side. (c)–(d) The values 8 and 7 are placed into to high side. (e) The values 1 and 8 are swapped, and the low side grows. (f) The values 3 and 7 are swapped, and the low side grows. (g)–(h) The high side of the partition grows to include 5 and 6, and the loop terminates. (i) Line 7 swaps the pivot element so that it lies between the two sides of the partition, and line 8 returns the pivot’s new index.

The final two lines of PARTITION finish up by swapping the pivot with the leftmost element greater than x , thereby moving the pivot into its correct place in the partitioned array, and then returning the pivot’s new index. The output of PARTITION now satisfies the specifications given for the divide step. In fact, it satisfies a slightly stronger condition:

after line 3 of QUICKSORT, $A[q]$ is strictly less than every element of $A[q + 1 : r]$.

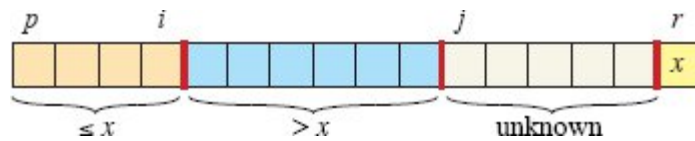


Figure 7.2 The four regions maintained by the procedure PARTITION on a subarray $A[p : r]$. The tan values in $A[p : i]$ are all less than or equal to x , the blue values in $A[i + 1 : j - 1]$ are all greater than x , the white values in $A[j : r - 1]$ have unknown relationships to x , and $A[r] = x$.

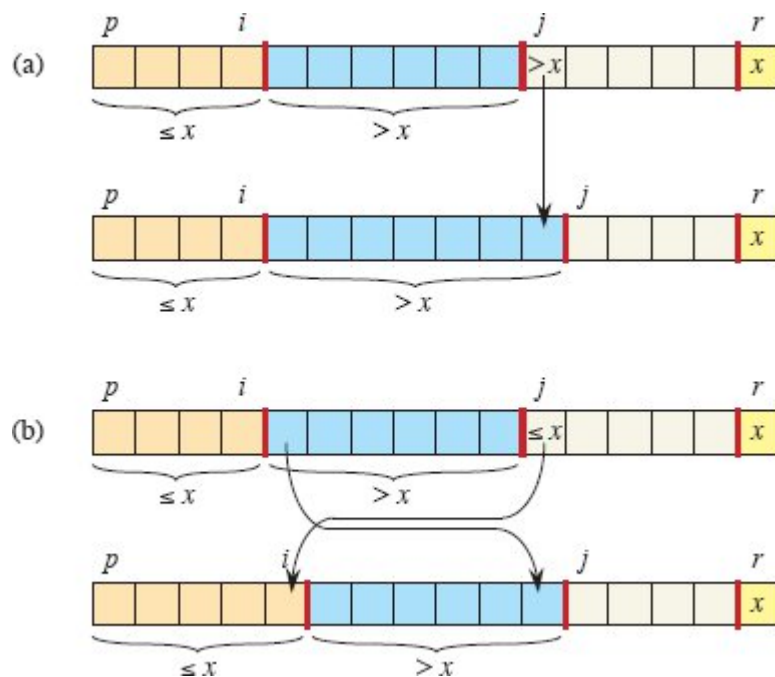


Figure 7.3 The two cases for one iteration of procedure PARTITION. **(a)** If $A[j] > x$, the only action is to increment j , which maintains the loop invariant. **(b)** If $A[j] \leq x$, index i is incremented, $A[i]$ and $A[j]$ are swapped, and then j is incremented. Again, the loop invariant is maintained.

Exercise 7.1-3 asks you to show that the running time of PARTITION on a subarray $A[p : r]$ of $n = r - p + 1$ elements is $\Theta(n)$.

Exercises

7.1-1

Using Figure 7.1 as a model, illustrate the operation of PARTITION on the array $A = \langle 13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11 \rangle$.

7.1-2

What value of q does PARTITION return when all elements in the subarray $A[p : r]$ have the same value? Modify PARTITION so that $q = \lfloor (p + r)/2 \rfloor$ when all elements in the subarray $A[p : r]$ have the same value.

7.1-3

Give a brief argument that the running time of PARTITION on a subarray of size n is $\Theta(n)$.

7.1-4

Modify QUICKSORT to sort into monotonically decreasing order.

7.2 Performance of quicksort

The running time of quicksort depends on how balanced each partitioning is, which in turn depends on which elements are used as pivots. If the two sides of a partition are about the same size—the partitioning is balanced—then the algorithm runs asymptotically as fast as merge sort. If the partitioning is unbalanced, however, it can run asymptotically as slowly as insertion sort. To allow you to gain some intuition before diving into a formal analysis, this section informally investigates how quicksort performs under the assumptions of balanced versus unbalanced partitioning.

But first, let's briefly look at the maximum amount of memory that quicksort requires. Although quicksort sorts in place according to the definition on page 158, the amount of memory it uses—aside from the array being sorted—is not constant. Since each recursive call requires a constant amount of space on the runtime stack, outside of the array being sorted, quicksort requires space proportional to the maximum depth of the recursion. As we'll see now, that could be as bad as $\Theta(n)$ in the worst case.